GateTunnel

System Design Description

Abstract

This document describes the design and implementation of the GateTunnel support system of Arrowhead 5.2.2 (`ah5-GateTunnel-java-spring`).

Contents

1 Overview **3**

2 Implementation **4**
2.1 Implementation language and tools 4
2.2 Functional properties implementation 4
2.3 Non functional properties implementation 5
2.4 Deployment 5
2.5 Local modifications 6
2.6 Verification 6

3 Services **7**

4 References **7**

5 Revision History **9**
5.1 Amendments 9
5.2 Quality Assurance 9

1 Overview

This document describes the GateTunnel system implementation, which realizes the GateTunnel SysD [1]. GateTunnel carries the inter-cloud service calls negotiated by the Gatekeepers: a consumer session listens locally and forwards each request over the relay to the provider cloud's GateTunnel, which calls the provider. In Section 2, we describe implementation details of the system.

2 Implementation

This implementation realizes the GateTunnel SysD in `ah5-GateTunnel-java-spring`. It follows the Arrowhead Java-Spring system pattern of the support systems (as Blacklist and EventHandler). The Java sources, the private copy of the Arrowhead common library (`vendor/`), tests, images and deployment definition are owned by this folder. The earlier native implementation (JDK `HttpsServer`, SQLite, package `eu.arrowhead.port`) has been replaced; the wire contracts are unchanged. `legacy/` preserves an older implementation and is excluded from the build.

2.1 Implementation language and tools

- Java 21, Spring Boot 3.4 (web, security, data JPA, Quartz), the vendored Arrowhead common library (common-utils, data-transfer-objects), Bouncy Castle, Eclipse Paho MQTT v3 (relay client) and the RabbitMQ AMQP client (sysMonitor). Maven builds the Spring Boot JAR `arrowhead-gatetunnel-5.2.2.jar`; the POM inherits only the Spring Boot parent.
- Storage: a private MySQL 8.4 schema `ah_gatetunnel` through JPA (MariaDB driver, `sslMode=verify-full`, `ddl-auto=validate`).

2.2 Functional properties implementation

The package `eu.arrowhead.gatetunnel` holds the following components:

Component	Responsibility
`GateTunnelSystemInfo`, `init/`	Registered system and services; at start, the system certificate must be `GateTunnel.<cloud>.<organisation>.arrowhead.eu` with profile `sy`. Registration itself is done by the common `ApplicationInitListener`.
`api/http/`	Controllers of gateTunnelConnection, gateTunnelKey, gateTunnelManagement, generalManagement, echo and liveness. Each path is served under `/gatetunnel`, `/gateway` and without base path.
`security/GateTunnelCallerFilter`	Caller rules after the common authentication filter: certificate profile `sy`/`op`; management paths for LocalCloudManager and Sysop; other operations for Gatekeeper, DynamicServiceOrchestration, SimpleStoreServiceOrchestration, LocalCloudManager and Sysop; inter-cloud sessions for the local Gatekeeper only.
`service/InterCloudSessionService`	Inter-cloud relay sessions (in memory): provider side executes relayed requests within the negotiated operations; consumer side runs a mutual-TLS tunnel listener per session in the pool 8200–8299 and forwards over the relay. Ported verbatim in behaviour from 5.2.2.
`service/SessionSecurity`, `PeerNames`, `validation/OperationScope`	AH-GT-1 key exchange, sealing and replay window; explicit cross-organisation bindings; scope of a relayed request. Byte-for-byte compatible with the Go and Rust GateTunnels.
`service/RelayClient`	Paho MQTT client with the GateTunnel's own relay identity (`RELAY_*`); topics `arrowhead/relay/<cloud>/gatetunnel/in/<sender CN>/<id>`.
`service/ProviderClient`	Calls the negotiated provider with the own identity token, pinned to the provider's certificate CN, renewing a refused token once.

Component	Responsibility
`service/ControlSession Service`, `jpa/`	4.x-compatible control sessions (control port pool 8000–8099).
`service/GateTunnelJobs`	Scheduled relay (re)connection (10 s) and session expiry (60 s).

2.2.1 Data handled and stored

Storage: MySQL schema `ah_gatetunnel`:

Tables	Content
`session`	Control sessions: session and queue identifiers, peer name, service definition, reserved port, state, parties as JSON. Closed sessions are removed after `SESSION_MAX_AGE_SECONDS`, which frees their ports.
`logs`	Log entries (Log4j2 JDBC appender), read by general-Management `get-log`.

Inter-cloud sessions and their keys are kept in memory only. The interfaces exchange JSON objects; their structure is specified in the IDD documents. Error responses use the Arrowhead error body with the 5.2.2 status codes.

2.3 Non functional properties implementation

2.3.1 Security

The web server requires client certificates (PKCS12 key and trust stores) and checks them against the cloud CA's CRL. The common `OutsourcedFilter` verifies the identity token with Authentication and binds it to the client certificate; the caller filter then applies the rules above. Tunnel listeners and provider calls use the same identity and CRL outside the web server. `SecurityProfileGuard` refuses a configuration below the secure profile. The database account has data-manipulation privileges only.

2.3.2 Power management

None; the system is a server process without power states. The `sysMonitor power` observation always reports line supply.

2.3.3 Internal monitoring

Logs go to stdout, a log file and the `logs` table. The common `SysMonitorPublisher` publishes the `sysMonitor` observations; the health contribution is `degraded` while the relay link is down.

2.3.4 Configuration

`config/application.properties`; the 5.2.2 environment names (`LOCAL_CLOUD`, `ADVERTISED_ADDRESS`, `NEIGHBOR_CLOUDS`, `FEDERATION_*`, `RELAY_*`, port pools, session times) are placeholders in it. Secrets are Compose secrets read through `configtree:/run/secrets/`.

2.4 Deployment

The local `Dockerfile` builds this folder and runs `arrowhead-gatetunnel-standalone:5.2.2` as UID 10001 with a read-only root filesystem. `compose.yaml` adds `gatetunnel-db` (MySQL 8.4.11, TLS required, internal network; `deployment/database/init.sql` and `least-privilege.sh`) and joins an operator-selected cloud network. The API uses port 8453; the health check is a TCP probe of it (the former plain-HTTP

health port 8481 is gone). Certificates and credentials are mounted individually. Existing test-cloud deployments are not changed by this packaging.

ServiceRegistry is the bootstrap endpoint; Authentication and the orchestrator are found through it. Federation requires the organisation MQTT relay, configured neighbours, the relay identity and organisation trust. The Compose default is AH-GT-1 with explicit legacy-mTLS compatibility; peers must agree, and there is never a silent downgrade.

2.4.1 Ownership and build boundary

The Arrowhead common library is a private vendored source copy (`vendor/PROVENANCE.json`); security fixes must be propagated to every vendored copy. There is no shared runtime process, database or Maven parent besides Spring Boot's. Build with Java 21 and `mvn -B clean verify` (`scripts/verify.sh` also checks the JAR contents), or with the local Docker build.

2.5 Local modifications

Notes on this implementation in the workspace:

- Java-Spring port of the native implementation; inter-cloud sessions for DynamicServiceOrchestration (local extension).
- Services registered with version `5.2.2`; generalManagement adds `get-log` and `get-config`.
- `/health` now requires an identity token; unprefixed path aliases are served.

2.6 Verification

JUnit tests: the AH-GT-1 reference vector (key exchange, key derivation, sealing, replay), peer names, operation scope, TLS pinning and token renewal, the refused insecure start, and a MockMvc contract test of every served path and method through the real authentication filter. Integration tests against containers: the application against a fresh database container (schema validated with DML-only grants), and an AH-GT-1 round trip between two relay identities over an MQTT broker with mutual TLS. A live federation run with the Go and Rust GateTunnels has not been repeated for the Spring build.

3 Services

The implementation of the services is based on the SD and IDD documents referenced below.

Services produced	**SysD ref**	**SD ref**	**IDD ref**
`gateTunnelConnection`	[1]	[2]	[3]
`gateTunnelKey`	[1]	[4]	[5]
`gateTunnelManagement`	[1]	[6]	[7]
`sysMonitor`	[1]	[8]	[9]
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
serviceDiscovery	[10]	[11]	–
identity	[12]	[13]	–
inter-cloud messages	Relay (MQTT)	–	–
negotiated service	Providers	–	–
sysMonitor (publisher)	AMQP broker (SysMonitor)	–	–

4 References

[1] J. Delsing, “GateTunnel – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-GateTunnel-java-spring/documentation/v5.2.2/GateTunnel-v5.2.2-SysD.pdf.

[2] ——, “gateTunnelConnection – Service Description (GateTunnel),” Eclipse Arrowhead, SD 5.2.2, 2026, gateTunnel-v5.2.2-gateTunnelConnection-SD.pdf.

[3] ——, “gateTunnelConnection HTTP/TLS/JSON – Interface Design Description (GateTunnel),” Eclipse Arrowhead, IDD 5.2.2, 2026, gateTunnel-v5.2.2-gateTunnelConnection-HTTP-IDD.pdf.

[4] ——, “gateTunnelKey – Service Description (GateTunnel),” Eclipse Arrowhead, SD 5.2.2, 2026, gateTunnel-v5.2.2-gateTunnelKey-SD.pdf.

[5] ——, “gateTunnelKey HTTP/TLS/JSON – Interface Design Description (GateTunnel),” Eclipse Arrowhead, IDD 5.2.2, 2026, gateTunnel-v5.2.2-gateTunnelKey-HTTP-IDD.pdf.

[6] ——, “gateTunnelManagement – Service Description (GateTunnel),” Eclipse Arrowhead, SD 5.2.2, 2026, gateTunnel-v5.2.2-gateTunnelManagement-SD.pdf.

[7] ——, “gateTunnelManagement HTTP/TLS/JSON – Interface Design Description (GateTunnel),” Eclipse Arrowhead, IDD 5.2.2, 2026, gateTunnel-v5.2.2-gateTunnelManagement-HTTP-IDD.pdf.

[8] ——, “sysMonitor – Service Description (GateTunnel),” Eclipse Arrowhead, SD 5.2.2, 2026, gateTunnel-v5.2.2-sysMonitor-SD.pdf.

[9] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (GateTunnel),” Eclipse Arrowhead, IDD 5.2.2, 2026, gateTunnel-v5.2.2-sysMonitor-AMQP-IDD.pdf.

[10] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-SysD.pdf.

[11] ——, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-serviceDiscovery-SD.pdf.

[12] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-SysD.pdf.

[13] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-identity-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-25	5.2.2	Independent Java build, private helpers and state, per-system deployment and configuration.	Codex (draft)
3	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated.	Claude Opus 5.5 (draft)
4	2026-11-08	5.2.2	Implementation sections updated for the Java-Spring port (common library, MySQL via JPA, Spring beans for the relay and AH-GT-1); contracts unchanged.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	